

Piper 机械臂双 ACT 长程任务

推理代码修改与安全实践手册

LeRobot 环境 / 双技能组合 / 受控切换 / Piper 真实机械臂

技能 A

→

人工确认

→

技能 B

→

完成任务

安全声明

任何软件代码都不能保证真实机械臂绝对安全。本手册通过默认 Dry Run、动作范围限制、单步变化限制、阶段超时、人工确认和异常停止降低风险，但物理急停、现场监护和安全场景仍然是必需条件。

适用对象：已完成 Piper 单 ACT 采集、训练和部署实验的学生

版本：课程学生实践版 · 2026 年 7 月

开始实验前：必须阅读的安全要求

红线要求：任意一项不满足时，不得让机械臂执行模型动作

急停或断电装置可立即触及；机械臂运动范围内无人手、线缆、玻璃、液体和尖锐物；两个 ACT 模型均已单独通过低速部署；相机、动作单位和模型路径已由同伴或教师复核。

- ☐ 第一次运行必须保持 EXECUTE_ROBOT=False，只检查模型加载、相机、观测和预测动作。
- ☐ 只有完成 Dry Run、空场景测试和教师检查后，才可把 EXECUTE_ROBOT 改为 True。
- ☐ 技能 A 完成后不自动启动技能 B。程序进入保持状态，由操作人员检查场景后按 Enter 确认。
- ☐ 出现抖动、方向错误、异常加速、夹爪夹人风险或接近限位时，立即按 Ctrl+C；若动作仍未停止，立即使用物理急停或断电。
- ☐ 不得用手对抗带扭矩的机械臂，不得在模型运行时手动摆正机械臂或物体。
- ☐ 软件限幅只能降低风险，不能检测桌面碰撞、物体卡死或人与机械臂接触。全程必须有人监护。

代码颜色说明

显示方式	含义	学生操作
浅灰色代码	课程提供的已有代码，包含详细注释。	阅读和理解，不要随意改动。
黄色代码	完成本实验必须填写或修改的内容。	按本组模型、相机和时长修改。
橙色代码	提高部分，可在基础版本稳定后尝试。	选做；修改前先备份。

目录

1. 实验目标与双 ACT 实现思路
 2. LeRobot 中 ACT 推理机制
 3. 安全设计与代码审查结论
 4. 实验前准备与单技能验收
 5. 双 ACT 推理脚本的分步实现
 6. 学生需要完成的修改
 7. 四类任务的技能拆分建议
 8. 安全调试顺序与常见问题
 9. 实验验收与提交内容
- 附录 A. 完整安全代码模板

版本说明

本手册依据课程当前使用的 LeRobot 和 lerobot_robot_piper 接口编写。实验当天不要升级 LeRobot、Piper 插件或 PyTorch；若课程固定版本的导入路径略有不同，以教师发放环境为准。

1. 实验目标与双 ACT 实现思路

一个 ACT 模型直接学习完整长程轨迹时，前半段误差会改变后半段的起始状态，导致后半段成功率下降。双 ACT 方案把长任务拆成两个较短技能，并通过一个简单状态机进行组合。

方案	模型数量	需要修改代码	特点
单 ACT 基础方案	1 个	否	直接学习整条任务，流程简单，但长程误差容易累积。
双 ACT 进阶方案	2 个	是	每个模型学习一个短技能；交接状态更容易分析和补采。

本实验只修改推理组织逻辑，不修改 ACT 的网络结构、Transformer、视觉骨干、损失函数或训练代码，也不修改 Piper CAN 通信底层。

- 分别加载技能 A 和技能 B 的 ACT checkpoint。
- 为两个模型分别加载预处理器和后处理器。
- 使用 Stage 状态变量选择当前执行的模型。
- 技能切换前调用 `policy.reset()`，清除动作队列或时序集成状态。
- 在模型动作发送到 Piper 前增加安全过滤。
- 技能 A 后进入人工确认交接，避免在 A 失败时自动执行 B。

2. LeRobot 中 ACT 推理机制

2.1 需要阅读的代码位置

文件	需要理解的内容
src/lerobot/policies/act/modeling_act.py	ACTPolicy.reset()、select_action()、predict_action_chunk()。
src/lerobot/policies/act/configuration_act.py	chunk_size、n_action_steps、temporal_ensemble_coeff。
src/lerobot/policies/act/processor_act.py	状态与动作归一化、反归一化。
examples/tutorial/act/act_using_example.py	官方单策略推理链：观测、处理器、模型、动作发送。
lerobot_robot_piper/piper.py	Piper 的观测读取、动作键映射和硬件范围裁剪。

2.2 为什么阶段切换必须调用 reset()

ACT 一次预测一个动作块。select_action() 会在内部动作队列为空时生成新动作块，再逐步返回动作；启用时序集成时，策略还保存跨时间的集成状态。reset() 会清空这些内部状态。

```
def reset(self):
    # 环境或任务阶段重置时清空内部状态。
    if self.config.temporal_ensemble_coeff is not None:
        self.temporal_ensemble.reset()
    else:
        self._action_queue = deque([], maxlen=self.config.n_action_steps)

def select_action(self, batch):
    # 动作队列为空时才重新预测一个动作块。
    if len(self._action_queue) == 0:
        actions = self.predict_action_chunk(batch)
        self._action_queue.extend(actions.transpose(0, 1))
    return self._action_queue.popleft()
```

正确做法

不要直接访问或清空私有变量 _action_queue。开始一轮任务时重置两个模型；由技能 A 切换到技能 B 前再次调用 skill_b.policy.reset()。

2.3 为什么两个模型必须分别加载处理器

预处理器包含输入归一化和设备迁移，后处理器负责将模型动作恢复到机器人动作范围。两个技能的数据统计量可能不同，因此必须分别保存 pre_a/post_a 和 pre_b/post_b。

```
原始 Piper 观测
-> 技能对应的 preprocess
-> ACTPolicy.select_action
-> 技能对应的 postprocess
-> make_robot_action
-> 安全过滤
```

-> Piper.send_action

3. 安全设计与代码审查结论

对原版模板进行审查后，重点修正了以下风险。

原风险	本版修改
固定时间到达后直接切换技能 B	技能 A 后进入 WAIT_CONFIRM，必须由操作人员检查并确认。
直接发送模型输出，缺少单步速度限制	增加有限数检查、绝对范围裁剪和每周期最大变化限制。
第一次运行就可能驱动机械臂	默认 EXECUTE_ROBOT=False，先执行 Dry Run。
阶段或总任务可能无限运行	分别设置技能时长、交接确认超时和总任务超时。
异常退出只断开连接	finally 中尽力发送当前姿态保持指令，再断开；仍要求物理急停。
模型路径错误可能在带电后才发现	连接机械臂之前先检查模型目录和 config.json。

3.1 Piper 插件已有保护与仍需补充的保护

课程当前 Piper 插件会把归一化关节命令限制到允许范围，并在转换为硬件角度后再次裁剪到硬件最小值与最大值。该保护能够减少越过绝对关节范围的风险，但不能限制相邻控制周期之间的动作跳变，也不能感知桌面碰撞。

本版增加的安全过滤

关节命令限制在 -95 到 95；夹爪限制在 0 到 100；每个 15 Hz 控制周期内，关节最多变化 2 个归一化单位、夹爪最多变化 4 个单位；出现 NaN、Inf、动作键缺失或观测键缺失时立即停止。

不能保证的事项

软件无法判断夹爪是否夹住人、物体是否卡住、末端是否会碰桌面，也无法在 CAN 通信完全失效时发送保持指令。因此“保证不会损伤机械臂”不能只依靠代码，必须配合物理急停、合理场景、低风险测试和人工监护。

4. 实验前准备与单技能验收

4.1 激活固定环境

```
conda activate lerobot
cd ~/lerobot

python -c "import lerobot; print(lerobot.__file__)"
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

```
python -c "import lerobot_robot_piper; print('piper plugin: OK')"
```

不要升级

实验当天不要执行 `pip install -U lerobot`，不要切换 Git 分支，也不要单独更新 Piper 插件。

4.2 检查 CAN 与摄像头

```
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up
ip -details link show can0

v4l2-ctl --list-devices
lerobot-find-cameras opencv
```

front、wrist 的名称、摄像头编号、640×480 分辨率和 30 fps 必须与两个技能数据采集时一致。

4.3 双 ACT 组合前的强制验收

- ☐ 技能 A 单独部署成功，并能稳定停到交接状态。
- ☐ 手动把机械臂和场景摆成交接状态后，技能 B 单独部署成功。
- ☐ 两个数据集的相机名称、机器人动作定义、`include_gripper` 和 `use_degrees` 完全一致。
- ☐ 技能 A 失败时，继续执行技能 B 不会导致机械臂进入危险区域。
- ☐ 模型 A 和 B 的 `pretrained_model` 目录完整，且分别保存了处理器配置。

5. 双 ACT 推理脚本的分步实现

不要修改 `modeling_act.py` 或 LeRobot 原有 `rollout` 脚本。在 `~/lerobot/examples/student/` 下新建 `dual_act_piper_rollout.py`。这样即使学生代码出错，也不会破坏训练和原始推理命令。

```
conda activate lerobot
cd ~/lerobot
mkdir -p examples/student
cp /课程发放目录/dual_act_safe_template_cn.py \
  examples/student/dual_act_piper_rollout.py
```

5.1 导入接口：已有代码，不需要修改

```
import math          # 检查 NaN 和 Inf。
import select        # 非阻塞读取终端确认键。
import sys           # 读取标准输入。
import time          # 控制周期、阶段时长和超时。

from dataclasses import dataclass
from enum import Enum, auto
from pathlib import Path
from typing import Any

import torch

from lerobot.cameras.opencv import OpenCVCameraConfig
from lerobot.datasets.utils import hw_to_dataset_features
```

```

from lerobot.policies import make_pre_post_processors
from lerobot.policies.act import ACTPolicy
from lerobot.policies.utils import build_inference_frame, make_robot_action
from lerobot_robot_piper.config_piper import PiperConfig
from lerobot_robot_piper.piper import Piper

```

5.2 学生配置区：黄色行必须修改

```

POLICY_A_PATH = Path("/绝对路径/act_skill_a/pretrained_model") # 学生修改
POLICY_B_PATH = Path("/绝对路径/act_skill_b/pretrained_model") # 学生修改

FRONT_CAMERA_INDEX = 0 # 学生修改
WRIST_CAMERA_INDEX = 2 # 学生修改

SKILL_A_MAX_SECONDS = 35.0 # 学生修改
SKILL_B_MAX_SECONDS = 35.0 # 学生修改

# 第一次运行必须保持 False；教师检查后最后再改为 True。
EXECUTE_ROBOT = False # 学生最后修改

```

路径建议

首次使用绝对路径。模型目录应直接指向 pretrained_model，而不是其上级 checkpoint 目录。

5.3 安全参数：已有代码，不允许学生放宽

```

CONTROL_FPS = 15 # 与数据集动作频率一致。
TOTAL_MAX_SECONDS = 100.0 # 整条任务总超时。
HANDOFF_CONFIRM_TIMEOUT_S = 30.0 # 交接确认超时。
STARTUP_COUNTDOWN_S = 5 # 正式执行前倒计时。

JOINT_ABS_MIN = -95.0
JOINT_ABS_MAX = 95.0
GRIPPER_ABS_MIN = 0.0
GRIPPER_ABS_MAX = 100.0
MAX_JOINT_DELTA_PER_STEP = 2.0
MAX_GRIPPER_DELTA_PER_STEP = 4.0

```

这些数值是课程的初始保守设置。若某个任务确实需要更大运动范围，应由教师在空场景下重新评估，不允许学生为了提高速度自行放宽。

5.4 加载两个模型及独立处理器

```

@dataclass
class ACTBundle:
    # 一个 Bundle 始终把模型和自己的处理器绑定在一起，
    # 防止误用另一技能的归一化统计量。
    name: str
    policy: ACTPolicy
    preprocess: Any
    postprocess: Any

def load_act_bundle(name: str, model_path: Path) -> ACTBundle:
    validate_model_path(model_path, name)
    policy = ACTPolicy.from_pretrained(str(model_path)).to(DEVICE)
    policy.eval()

```

```

preprocess, postprocess = make_pre_post_processors(
    policy.config,
    pretrained_path=str(model_path),
    preprocessor_overrides={
        "device_processor": {"device": str(DEVICE)},
    },
)
policy.reset()
return ACTBundle(name, policy, preprocess, postprocess)

```

policy.eval() 关闭训练模式；torch.inference_mode() 关闭梯度计算。pretrained_path 指向各自 checkpoint，确保处理器与对应模型配套。

5.5 Piper、相机与数据特征

```

cameras = {
    "front": OpenCVCameraConfig(
        index_or_path=FRONT_CAMERA_INDEX,
        width=640, height=480, fps=30,
    ),
    "wrist": OpenCVCameraConfig(
        index_or_path=WRIST_CAMERA_INDEX,
        width=640, height=480, fps=30,
    ),
}

config = PiperConfig(
    can_interface="can0",
    bitrate=1_000_000,
    include_gripper=True,
    use_degrees=False,
    cameras=cameras,
)
robot = Piper(config)

action_features = hw_to_dataset_features(
    robot.action_features, "action"
)
observation_features = hw_to_dataset_features(
    robot.observation_features, "observation"
)
dataset_features = {**action_features, **observation_features}

```

禁止混用动作单位

本课程数据采集使用 use_degrees=False。若模型训练时的设置不同，不得通过简单改一行强行部署；必须先确认数据动作范围与 Piper 配置完全一致。

5.6 标准推理链：已有代码，不需要修改

```

def infer_action(bundle, raw_observation, dataset_features):
    # 1. 将摄像头、关节和夹爪观测构造造成 LeRobot 推理帧。
    frame = build_inference_frame(
        observation=raw_observation,
        ds_features=dataset_features,
        device=DEVICE,
    )

```



```

)

# 2. 使用当前技能自己的统计量归一化输入。
batch = bundle.preprocess(frame)

# 3. ACT 返回动作队列中的下一步动作。
with torch.inference_mode():
    action_tensor = bundle.policy.select_action(batch)

# 4. 反归一化，并转换成 Piper 动作字典。
action_tensor = bundle.postprocess(action_tensor)
robot_action = make_robot_action(action_tensor, dataset_features)
return {k: float(v) for k, v in robot_action.items()}

```

顺序不可改变

跳过 preprocess 或 postprocess 可能把归一化动作直接发送给机械臂；绕过 make_robot_action 会造成动作键与 Piper 接口不匹配。

5.7 动作安全过滤：已有代码，不需要修改

```

def safe_filter_action(proposed_action, observation, expected_keys):
    # 模型必须输出 Piper 所需的全部动作键。
    missing = expected_keys - proposed_action.keys()
    if missing:
        raise KeyError(f"模型动作缺少必要键: {sorted(missing)}")

    safe_action = {}
    for key in sorted(expected_keys):
        target = float(proposed_action[key])
        current = float(observation[key])

        # NaN 或 Inf 不能进入机械臂控制接口。
        if not math.isfinite(target) or not math.isfinite(current):
            raise ValueError(f"检测到非有限数值: {key}")

        if key == "gripper.pos":
            target = clamp(target, 0.0, 100.0)
            max_delta = MAX_GRIPPER_DELTA_PER_STEP
        else:
            target = clamp(target, -95.0, 95.0)
            max_delta = MAX_JOINT_DELTA_PER_STEP

        # 目标与当前姿态的差值每周期最多变化 max_delta。
        delta = clamp(target - current, -max_delta, max_delta)
        safe_action[key] = current + delta

    return safe_action

```

安全过滤发生在 postprocess 和 make_robot_action 之后、send_action 之前。这样过滤器处理的是与 Piper 接口一致的实际动作字典。

5.8 阶段切换与人工确认

```

class Stage(Enum):
    SKILL_A = auto()
    WAIT_CONFIRM = auto()

```

```

SKILL_B = auto()
DONE = auto()
ABORTED = auto()

def should_switch_to_b(observation, elapsed_s):
    del observation
    # 基础版只决定何时进入“人工确认”，不会自动执行 B。
    return elapsed_s >= SKILL_A_MAX_SECONDS

# 学生选做：用“最短时间 + 已验证状态条件”替换上一行。

```

固定时间到达后进入 WAIT_CONFIRM。等待期间程序读取当前姿态并持续发送保持指令；只有按 Enter 后才重置技能 B 并启动。按 q 或等待超时则终止任务。

为什么不自动切换

若技能 A 抓取失败或物体落在错误位置，自动执行技能 B 可能导致碰撞。人工确认是本课程的默认安全边界，也是最容易理解和调试的实现。

5.9 主循环中的关键顺序

```

if stage is Stage.SKILL_A:
    observation = robot.get_observation()
    proposed = infer_action(skill_a, observation, dataset_features)
    safe_action = safe_filter_action(
        proposed, observation, expected_action_keys
    )
    send_or_preview(robot, safe_action)

    if should_switch_to_b(observation, stage_elapsed):
        stage = Stage.WAIT_CONFIRM

elif stage is Stage.WAIT_CONFIRM:
    confirmed = wait_for_handoff_confirmation(
        robot, expected_action_keys
    )
    if not confirmed:
        stage = Stage.ABORTED
    else:
        skill_b.policy.reset() # 清空技能 B 的动作内部状态。
        stage = Stage.SKILL_B

elif stage is Stage.SKILL_B:
    observation = robot.get_observation()
    proposed = infer_action(skill_b, observation, dataset_features)
    safe_action = safe_filter_action(
        proposed, observation, expected_action_keys
    )
    send_or_preview(robot, safe_action)

```

6. 学生需要完成的修改

编号	修改项	要求	完成后检查
1	POLICY_A_PATH	填写技能 A pretrained_model 的绝对路径。	目录存在且含 config.json。
2	POLICY_B_PATH	填写技能 B pretrained_model 的绝对路径。	两个路径不能误写为同一模型。
3	摄像头编号	根据 lerobot-find-cameras 修改 front/wrist。	画面方向和训练时一致。
4	技能最大时长	根据单技能多次测试结果填写。	略大于稳定完成时间，但不能过长。
5	EXECUTE_ROBOT	所有 Dry Run 和教师检查完成后最后改为 True。	急停在手边，空场景先测。
6（选做）	should_switch_to_b	增加简单状态条件，但保留人工确认。	条件失败时不得自动进入 B。

6.1 学生不应修改的部分

- ACTPolicy、processor、Piper CAN 底层和硬件限位代码。
- 安全过滤函数、总超时、交接确认超时和 finally 异常清理。
- use_degrees、include_gripper、相机名称和动作键映射。
- 为了“动作更快”而增大单步变化量或删除 Dry Run。

7. 四类任务的技能拆分建议

长程任务	技能 A	技能 B	推荐交接状态
桌面整理	香蕉放入盘子	积木放入收纳盒	香蕉已稳定放好；夹爪离开盘子并停在固定侧上方。
清障抓取	障碍物移到整理区	目标物抓取并放入容器	障碍物离开目标；目标清晰可见；机械臂停在固定中间姿态。
装配准备	底座放到中央工作区	零件和工具摆放	底座位置稳定；机械臂离开底座上方。
桌面清理	垃圾物放入垃圾盒	工具和目标物继续归位	垃圾已清除；剩余场景与技能 B 训练起点一致。

7.1 交接状态自检

- ☐ 技能 A 是否有明确、稳定、可重复的结束状态？

- ☐ 技能 B 的数据是否从相同交接状态开始采集？
- ☐ 交接时机械臂是否远离桌面、容器边缘和已放置物体？
- ☐ 技能 A 偶尔失败时，操作人员能否在人工确认阶段安全终止？
- ☐ 两个技能是否使用相同相机、动作单位和控制频率？

8. 安全调试顺序与常见问题

8.1 强制调试顺序

1. 运行 `python -m py_compile`，确认脚本没有语法错误。
2. 保持 `EXECUTE_ROBOT=False`，运行完整脚本，确认两个模型、处理器、摄像头和观测键正常。
3. 单独测试技能 A 和技能 B，记录完成时间和交接状态。
4. 保持 Dry Run，完整走一遍 A -> WAIT_CONFIRM -> B 的状态机。
5. 移除所有物体，在空场景中把 `EXECUTE_ROBOT=True`，测试限速、Ctrl+C 和人工确认。
6. 放置软质、轻质物体，在监护下低风险测试技能 A；确认后再测试 B。
7. 最后进行完整长程任务，每次只改变一个参数。

8.2 运行命令

```
conda activate lerobot
cd ~/lerobot

# 先做语法检查，不连接机械臂。
python -m py_compile examples/student/dual_act_piper_rollout.py

# 然后运行脚本。第一次必须是 EXECUTE_ROBOT=False。
python examples/student/dual_act_piper_rollout.py
```

8.3 常见问题

现象	优先处理
模型或处理器加载失败	检查 pretrained_model 是否完整；不要只复制单个权重文件。
技能 B 切换后动作异常	确认 B 使用自己的处理器；确认交接状态和动作单位一致。
切换时仍像技能 A	检查是否进入 SKILL_B 分支，并确认切换前调用 skill_b.policy.reset()。
Dry Run 正常，执行时动作仍过大	立即停止；检查 postprocess、use_degrees 和数据动作范围，禁止放宽限速。
动作持续被限速，任务变慢	先检查模型是否预测大跳变和交接状态；不要直接增大安全阈值。
人工确认后没有启动 B	检查终端是否为交互式 TTY；按 Enter 后观察 Stage 日志。
摄像头 KeyError/形状错误	front、wrist 名称、数量、宽高必须与训练一致。
Ctrl+C 后仍有残余运动	立即使用物理急停或断电；disconnect 不等于硬件制动。

9. 实验验收与提交内容

9.1 基础验收

- ☐ 技能 A 和技能 B 均能够独立完成。
- ☐ 脚本默认 Dry Run，并能正确加载两个模型与两套处理器。
- ☐ 模型动作经过安全过滤后才进入 send_action。
- ☐ 技能 A 后进入人工确认，按 q 可终止，按 Enter 才启动技能 B。
- ☐ Ctrl+C、阶段超时和总超时均能结束控制循环。
- ☐ 至少完成 3 次正式测试，记录完整成功次数和失败阶段。

9.2 提交内容

提交项	内容
双 ACT 推理脚本	保留详细注释；黄色必改项填写完整；不得删除安全代码。
模型说明	两个技能任务、数据量、checkpoint 和独立成功率。
交接状态说明	用文字或图片说明 A 结束与 B 开始的统一状态。
测试记录	至少 3 次，记录 A、交接、B 和完整任务结果。
结果分析	比较单 ACT 与双 ACT，分析成功率和主要失败阶段。

9.3 最终运行前检查

- ☐ 模型路径和相机编号已复核。
- ☐ can0 已启用，Piper 无硬件故障。
- ☐ 两个技能均已单独通过。
- ☐ 交接区域无障碍物，技能 B 起点与训练一致。
- ☐ EXECUTE_ROBOT 仅在教师批准后改为 True。
- ☐ 急停或电源开关在监护人员手边。
- ☐ 场景内无人员肢体、线缆、玻璃、液体和尖锐物。

参考代码位置

本手册的接口和安全说明依据课程固定版本及以下官方公开代码整理：

- Hugging Face LeRobot: src/lerobot/policies/act/modeling_act.py
- Hugging Face LeRobot: src/lerobot/policies/factory.py
- Hugging Face LeRobot: examples/tutorial/act/act_using_example.py
- AgRoboticsResearch/lerobot_robot_piper: lerobot_robot_piper/piper.py
- AgRoboticsResearch/lerobot_robot_piper: lerobot_robot_piper/config_piper.py

附录 A. 完整安全代码模板

浅灰色为课程提供代码；黄色为必须修改；橙色为选做。复制到

examples/student/dual_act_piper_rollout.py 后，先运行 py_compile，再进行 Dry Run。

```

"""Piper 双 ACT 安全推理模板。

必须在课程统一的 LeRobot conda 环境中运行。
脚本默认处于 DRY RUN：会连接机械臂并读取观测，但不会发送模型动作。
只有在教师检查通过后，才能将 EXECUTE_ROBOT 改为 True。
"""

import math
import select
import sys
import time
from dataclasses import dataclass
from enum import Enum, auto
from pathlib import Path
from typing import Any

import torch

from lerobot.cameras.opencv import OpenCVCameraConfig
from lerobot.datasets.utils import hw_to_dataset_features
from lerobot.policies import make_pre_post_processors
from lerobot.policies.act import ACTPolicy
from lerobot.policies.utils import build_inference_frame, make_robot_action
from lerobot_robot_piper.config_piper import PiperConfig
from lerobot_robot_piper.piper import Piper

# -----
# 学生配置区：只修改带有“学生修改”标记的行。
# -----
POLICY_A_PATH = Path("/绝对路径/act_skill_a/pretrained_model") # 学生修改
POLICY_B_PATH = Path("/绝对路径/act_skill_b/pretrained_model") # 学生修改

FRONT_CAMERA_INDEX = 0 # 学生修改
WRIST_CAMERA_INDEX = 2 # 学生修改

SKILL_A_MAX_SECONDS = 35.0 # 学生修改
SKILL_B_MAX_SECONDS = 35.0 # 学生修改

# 代码检查和第一次只读测试时必须保持 False。
# 只有模型路径、相机和场景全部通过教师检查后，才能改为 True。
EXECUTE_ROBOT = False # 学生最后修改

# -----
# 课程安全参数：未经教师批准，不得放宽这些数值。
# 本课程使用 use_degrees=False，因此关节动作是归一化百分比。
# -----
CONTROL_FPS = 15
TOTAL_MAX_SECONDS = 100.0
HANDOFF_CONFIRM_TIMEOUT_S = 30.0
STARTUP_COUNTDOWN_S = 5

JOINT_ABS_MIN = -95.0
JOINT_ABS_MAX = 95.0
GRIPPER_ABS_MIN = 0.0
GRIPPER_ABS_MAX = 100.0
MAX_JOINT_DELTA_PER_STEP = 2.0
MAX_GRIPPER_DELTA_PER_STEP = 4.0

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

@dataclass
class ACTBundle:
    """保存一个 ACT 模型，以及与该模型配套的预处理器和后处理器。"""

```

```

name: str
policy: ACTPolicy
preprocess: Any
postprocess: Any

class Stage(Enum):
    """双技能任务的有限状态机。"""

    SKILL_A = auto()
    WAIT_CONFIRM = auto()
    SKILL_B = auto()
    DONE = auto()
    ABORTED = auto()

def validate_model_path(path: Path, name: str) -> None:
    """在连接机械臂之前检查模型目录，避免带电后才发现路径错误。"""

    if not path.is_dir():
        raise FileNotFoundError(f"{name} 模型目录不存在: {path}")
    if not (path / "config.json").is_file():
        raise FileNotFoundError(f"{name} 缺少 config.json: {path}")

def load_act_bundle(name: str, model_path: Path) -> ACTBundle:
    """加载模型，并加载同一 checkpoint 保存的归一化处理器。"""

    validate_model_path(model_path, name)
    policy = ACTPolicy.from_pretrained(str(model_path)).to(DEVICE)
    policy.eval()

    preprocess, postprocess = make_pre_post_processors(
        policy.config,
        pretrained_path=str(model_path),
        preprocessor_overrides={
            "device_processor": {"device": str(DEVICE)},
        },
    )
    policy.reset()
    return ACTBundle(name, policy, preprocess, postprocess)

def build_robot() -> Piper:
    """使用与数据采集一致的参数创建 Piper 和两路摄像头。"""

    cameras = {
        "front": OpenCVCameraConfig(
            index_or_path=FRONT_CAMERA_INDEX,
            width=640,
            height=480,
            fps=30,
        ),
        "wrist": OpenCVCameraConfig(
            index_or_path=WRIST_CAMERA_INDEX,
            width=640,
            height=480,
            fps=30,
        ),
    }
    config = PiperConfig(
        can_interface="can0",
        bitrate=1_000_000,
        include_gripper=True,
        use_degrees=False,
        cameras=cameras,
    )
    return Piper(config)

def build_features(robot: Piper) -> dict[str, Any]:
    """将 Piper 硬件特征转换为 LeRobot 推理函数使用的数据特征。"""

```

```

action_features = hw_to_dataset_features(robot.action_features, "action")
observation_features = hw_to_dataset_features(
    robot.observation_features,
    "observation",
)
return {**action_features, **observation_features}

def infer_action(
    bundle: ACTBundle,
    raw_observation: dict[str, Any],
    dataset_features: dict[str, Any],
) -> dict[str, float]:
    """执行标准的 LeRobot 推理链：预处理 -> ACT -> 后处理。"""

    frame = build_inference_frame(
        observation=raw_observation,
        ds_features=dataset_features,
        device=DEVICE,
    )
    batch = bundle.preprocess(frame)

    with torch.inference_mode():
        action_tensor = bundle.policy.select_action(batch)

    action_tensor = bundle.postprocess(action_tensor)
    robot_action = make_robot_action(action_tensor, dataset_features)
    return {key: float(value) for key, value in robot_action.items()}

def clamp(value: float, lower: float, upper: float) -> float:
    """将数值限制在给定闭区间内。"""

    return max(lower, min(upper, value))

def safe_filter_action(
    proposed_action: dict[str, float],
    observation: dict[str, Any],
    expected_action_keys: set[str],
) -> dict[str, float]:
    """拒绝异常输出，并限制绝对范围和单步变化量。

    这是额外的软件保护层，不能替代 Piper 硬件限位、物理急停、
    无碰撞场景设计和现场监护。
    """

    missing = expected_action_keys - proposed_action.keys()
    if missing:
        raise KeyError(f"模型动作缺少必要键: {sorted(missing)}")

    safe_action: dict[str, float] = {}
    for key in sorted(expected_action_keys):
        target = float(proposed_action[key])
        current = float(observation[key])

        if not math.isfinite(target) or not math.isfinite(current):
            raise ValueError(f"检测到非有限数值: {key}")

        if key == "gripper.pos":
            target = clamp(target, GRIPPER_ABS_MIN, GRIPPER_ABS_MAX)
            max_delta = MAX_GRIPPER_DELTA_PER_STEP
        else:
            target = clamp(target, JOINT_ABS_MIN, JOINT_ABS_MAX)
            max_delta = MAX_JOINT_DELTA_PER_STEP

        delta = clamp(target - current, -max_delta, max_delta)
        safe_action[key] = current + delta

    return safe_action

def hold_current_action(

```



```

    observation: dict[str, Any],
    expected_action_keys: set[str],
) -> dict[str, float]:
    """生成等于当前测量姿态的位置指令，用于交接等待。"""

    return {key: float(observation[key]) for key in expected_action_keys}

def send_or_preview(robot: Piper, action: dict[str, float]) -> None:
    """只有显式开启 EXECUTE_ROBOT 后才向 Piper 发送动作。"""

    if EXECUTE_ROBOT:
        robot.send_action(action)
    else:
        preview = ", ".join(f"{k}={v:.2f}" for k, v in sorted(action.items()))
        print(f"[DRY RUN] {preview}")

def should_switch_to_b(observation: dict[str, Any], elapsed_s: float) -> bool:
    """判断技能 A 是否可以进入人工确认的交接阶段。"""

    del observation
    # 基础版：按时间进入交接，但仍必须由操作人员确认后才能启动技能 B。
    return elapsed_s >= SKILL_A_MAX_SECONDS

# 学生选做：将上一行替换为“最短时间 + 已验证的机器人状态条件”。
# 禁止只使用一个未经实测的关节阈值直接自动切换。

def wait_for_handoff_confirmation(
    robot: Piper,
    expected_action_keys: set[str],
) -> bool:
    """等待人工确认，同时持续刷新当前姿态保持指令。

    Enter 或 n + Enter：确认进入技能 B。
    q + Enter：终止整条任务。
    超过等待时间未确认时，程序自动终止，不会自行进入技能 B。
    """

    print("\n 请检查交接状态和机械臂姿态。")
    print("按 Enter（或 n + Enter）启动技能 B；按 q + Enter 终止。")
    deadline = time.perf_counter() + HANDOFF_CONFIRM_TIMEOUT_S

    while time.perf_counter() < deadline:
        observation = robot.get_observation()
        hold = hold_current_action(observation, expected_action_keys)
        send_or_preview(robot, hold)

        readable, _, _ = select.select([sys.stdin], [], [], 0.2)
        if readable:
            answer = sys.stdin.readline().strip().lower()
            if answer in {"", "n", "next"}:
                return True
            if answer in {"q", "quit", "stop"}:
                return False

    print("交接确认超时，任务将终止。")
    return False

def countdown() -> None:
    """正式执行前倒计时，让监护人员回到急停位置。"""

    if not EXECUTE_ROBOT:
        print("当前为 DRY RUN，不会向 Piper 发送动作。")
        return

    print("机械臂动作已启用，请将手放在急停附近。")
    for remaining in range(STARTUP_COUNTDOWN_S, 0, -1):
        print(f"{remaining} 秒后启动……")
        time.sleep(1.0)

```

```

def main() -> None:
    # 模型路径检查与模型加载发生在机械臂连接之前。
    skill_a = load_act_bundle("技能 A", POLICY_A_PATH)
    skill_b = load_act_bundle("技能 B", POLICY_B_PATH)
    robot = build_robot()

    stage = Stage.SKILL_A
    stage_start = time.perf_counter()
    task_start = stage_start

    try:
        robot.connect()
        dataset_features = build_features(robot)
        expected_action_keys = set(robot.action_features.keys())

        # 在运行模型前检查 Piper 是否返回了所有动作对应的状态键。
        first_observation = robot.get_observation()
        missing_obs_keys = expected_action_keys - first_observation.keys()
        if missing_obs_keys:
            raise KeyError(
                "机械臂观测缺少动作别名键: "
                f"{sorted(missing_obs_keys)}"
            )

        # 每次新任务都必须清空两个 ACT 的动作队列或时序集成状态。
        skill_a.policy.reset()
        skill_b.policy.reset()
        countdown()

        while stage not in {Stage.DONE, Stage.ABORTED}:
            tick_start = time.perf_counter()
            total_elapsed = tick_start - task_start
            stage_elapsed = tick_start - stage_start

            # 总超时是最后一道逻辑保护，避免代码无限运行。
            if total_elapsed >= TOTAL_MAX_SECONDS:
                print("达到整条任务总超时，程序终止。")
                stage = Stage.ABORTED
                break

            if stage is Stage.SKILL_A:
                observation = robot.get_observation()
                proposed = infer_action(
                    skill_a,
                    observation,
                    dataset_features,
                )
                safe_action = safe_filter_action(
                    proposed,
                    observation,
                    expected_action_keys,
                )
                send_or_preview(robot, safe_action)

                if should_switch_to_b(observation, stage_elapsed):
                    stage = Stage.WAIT_CONFIRM
                    stage_start = time.perf_counter()

            elif stage is Stage.WAIT_CONFIRM:
                confirmed = wait_for_handoff_confirmation(
                    robot,
                    expected_action_keys,
                )
                if not confirmed:
                    stage = Stage.ABORTED
                    continue

            # 切换前重置技能 B，清除旧动作块或时序集成状态。
            skill_b.policy.reset()
            stage = Stage.SKILL_B
            stage_start = time.perf_counter()

```

```

elif stage is Stage.SKILL_B:
    observation = robot.get_observation()
    proposed = infer_action(
        skill_b,
        observation,
        dataset_features,
    )
    safe_action = safe_filter_action(
        proposed,
        observation,
        expected_action_keys,
    )
    send_or_preview(robot, safe_action)

    if stage_elapsed >= SKILL_B_MAX_SECONDS:
        stage = Stage.DONE

    # 尽量维持与数据采集一致的 15 Hz 控制频率。
    used_s = time.perf_counter() - tick_start
    sleep_s = max(0.0, 1.0 / CONTROL_FPS - used_s)
    time.sleep(sleep_s)

except KeyboardInterrupt:
    print("收到 Ctrl+C, 正在停止任务。")
    stage = Stage.ABORTED

except Exception as exc:
    print(f"异常触发安全停止: {exc}")
    stage = Stage.ABORTED
    raise

finally:
    # 尽力发送一次当前姿态保持指令，然后断开连接。
    # 此过程不能替代物理急停；通信故障时可能无法发送保持指令。
    if robot.is_connected:
        try:
            current = robot.get_observation()
            expected = set(robot.action_features.keys())
            send_or_preview(robot, hold_current_action(current, expected))
        except Exception as hold_error:
            print(f"无法发送最终保持指令: {hold_error}")
        finally:
            robot.disconnect()

    print(f"任务结束状态: {stage.name}")

if __name__ == "__main__":
    main()

```